

Setup & Environment Guide

NU-CCIT Capstone Starter

Contents

Introduction	1
1. Docker Install & Troubleshooting	2
1.1 Installing Docker Desktop	2
1.2 Windows-specific issues (WSL2)	2
1.3 Mac-specific issues	2
1.4 Port conflicts	3
2. MongoDB Atlas Setup Walkthrough	4
2.1 Create your account and cluster	4
2.2 Create a database user	4
2.3 Allow network access	4
2.4 Get your connection string	4
2.5 Verifying the connection	5
3. Flutter Environment Setup (per OS)	6
3.1 Windows	6
3.2 macOS	6
3.3 Linux	6
3.4 Emulator / device setup (all platforms)	7
3.5 Pointing the app at your backend	7
4. .env Reference	8
4.1 backend/.env	8
4.2 frontend/.env	8
4.3 Mobile (Flutter) config	8
4.4 General .env tips	8

Introduction

This guide covers environment setup for the four things that trip up most teams early in the project: Docker, MongoDB Atlas, Flutter, and the .env files. Read the section for whichever database option your team picked (Docker or Atlas) — you don't need both.

1. Docker Install & Troubleshooting

Docker is only required if your team is running MongoDB **locally** (Option B in the main README). If you're using MongoDB Atlas instead, skip to Section 2.

1.1 Installing Docker Desktop

1. Download Docker Desktop for your OS: <https://www.docker.com/products/docker-desktop>
2. Run the installer and restart your machine if prompted.
3. Launch Docker Desktop and wait for the whale icon in your system tray/menu bar to stop animating — that means the engine is running.
4. Verify it works:

```
docker --version
docker compose version
```

Both commands should print version numbers. If either fails, Docker Desktop isn't running or wasn't added to your PATH — reopen the app and try again.

1.2 Windows-specific issues (WSL2)

Docker Desktop on Windows requires **WSL2** (Windows Subsystem for Linux 2) as its backend.

“WSL2 installation is incomplete” error:

```
wsl --install
wsl --update
```

Restart your machine after running these, then reopen Docker Desktop.

“Virtualization not enabled” error: Virtualization (Intel VT-x / AMD-V) needs to be turned on in your BIOS/UEFI settings. Restart your PC, enter BIOS setup (usually Del, F2, or F10 at boot), and look for a “Virtualization Technology” or “SVM Mode” setting to enable.

Docker is very slow on Windows: This almost always means your project files live on the Windows filesystem (C:\Users\...) instead of inside WSL2. Clone the repo inside your WSL2 filesystem (e.g. \\wsl\$\Ubuntu\home\yourname\) for much faster file syncing.

1.3 Mac-specific issues

Apple Silicon (M1/M2/M3) image compatibility: Some Docker images are built only for amd64 and will fail or run very slowly on Apple Silicon. If a container fails to start with an architecture-related error, add a platform flag to your compose service:

```
services:
  mongodb:
    platform: linux/amd64
```

“Cannot connect to the Docker daemon”: Docker Desktop isn’t running. Open the Docker Desktop app from Applications and wait for it to finish starting before running any docker command.

1.4 Port conflicts

If `docker compose up -d` fails with something like `port is already allocated`, another process is already using that port.

Find what’s using a port:

```
# Mac/Linux  
lsof -i :27018
```

```
# Windows (PowerShell)  
netstat -ano | findstr :27018
```

Fix options: - Stop the conflicting process (common culprits: a locally installed MongoDB service, or a previous container that didn’t shut down cleanly). - Or change the port mapping in `infra/docker-compose.yml`, e.g. change `"27018:27017"` to `"27019:27017"`, then update `MONGO_URI` in your backend `.env` to match.

Leftover containers from a previous run:

```
docker compose down  
docker ps -a          # check for containers still hanging around  
docker container prune
```

2. MongoDB Atlas Setup Walkthrough

Use this if your team picked Option A (no Docker) in the main README.

2.1 Create your account and cluster

1. Go to <https://www.mongodb.com/cloud/atlas/register> and sign up (Google/GitHub sign-in works too).
2. On the “Deploy your database” screen, choose the **M0 Free** tier.
3. Pick a cloud provider and region — pick whichever is geographically closest to you for lower latency; it doesn’t need to match your teammates.
4. Give the cluster a name (e.g. capstone-cluster) and click **Create**.

Cluster creation takes 1–3 minutes.

2.2 Create a database user

1. In the left sidebar, go to **Database Access** (under Security).
2. Click **Add New Database User**.
3. Choose **Password** authentication, set a username and a strong password (avoid special characters like @ or : — they break connection strings unless URL-encoded).
4. Under **Database User Privileges**, choose **Read and write to any database**.
5. Click **Add User**.

Write the username and password down somewhere safe — you’ll need them again in Section 2.4.

2.3 Allow network access

1. Go to **Network Access** (under Security).
2. Click **Add IP Address**.
3. For coursework, the simplest option is **Allow Access from Anywhere** (0.0.0.0/0) — click the button that fills this in automatically.
4. Click **Confirm**.

This is fine for a capstone project during development. For anything resembling production, restrict this to specific IPs instead.

2.4 Get your connection string

1. Go back to your cluster’s **Overview** page and click **Connect**.
2. Choose **Drivers**.
3. Select **Node.js** as the driver (matches the backend’s Mongoose/MongoDB client).
4. Copy the connection string. It looks like:

```
mongodb+srv://<username>:<password>@<cluster>.mongodb.net/?retryWrites=true&w=majo
```

5. Replace <username> and <password> with the credentials from Section 2.2, and add your database name before the ?:

mongodb+srv://myuser:mypassword@capstone-cluster.abcde.mongodb.net/capstonedb?retryWrites=true

6. Paste this as MONGO_URI in backend/.env.

2.5 Verifying the connection

Start the backend (`npm run dev`) and check the console for a MongoDB connection success message (this depends on how your team's `db.js/connection` file logs it). If it fails:

- Double-check the password doesn't contain unencoded special characters.
- Confirm Network Access includes your current IP or `0.0.0.0/0`.
- Confirm the database user has read/write privileges.

3. Flutter Environment Setup (per OS)

3.1 Windows

1. Download the Flutter SDK zip from <https://docs.flutter.dev/get-started/install/windows> and extract it to a path **without spaces**, e.g. C:\src\flutter.
2. Add C:\src\flutter\bin to your **PATH** environment variable (Settings → System → About → Advanced system settings → Environment Variables).
3. Install **Android Studio**, then open it once so it can install the Android SDK.
4. In Android Studio, go to **More Actions → SDK Manager** and note the **Android SDK Location** path (typically C:\Users\<you>\AppData\Local\Android\Sdk).
5. Run flutter doctor in a new terminal to confirm everything is set up. Fix any items it flags, in order, from top to bottom.

Common SDK path issue: if flutter doctor can't find the Android SDK, run:

```
flutter config --android-sdk "C:\Users\<you>\AppData\Local\Android\Sdk"
```

3.2 macOS

1. Download the Flutter SDK from <https://docs.flutter.dev/get-started/install/macos> or install via Homebrew:

```
brew install --cask flutter
```

2. Install **Xcode** from the App Store (required for iOS builds and the iOS Simulator).
3. After installing Xcode, run:

```
sudo xcodebuild -license accept  
sudo xcode-select --install
```

4. Install **Android Studio** for Android SDK/emulator support, same as Windows above.
5. Run flutter doctor and resolve anything it flags — a common one is accepting Android licenses:

```
flutter doctor --android-licenses
```

3.3 Linux

1. Download the Flutter SDK tarball from <https://docs.flutter.dev/get-started/install/linux> and extract it, e.g. to ~/development/flutter.
2. Add it to your PATH in ~/.bashrc or ~/.zshrc:

```
export PATH="$PATH:$HOME/development/flutter/bin"
```

3. Install Android Studio (via the [official tarball](#) or a package manager like snap: sudo snap install android-studio --classic).
4. Run flutter doctor and follow its instructions — Linux setups commonly need extra packages like clang, cmake, ninja-build, and libgtk-3-dev for desktop builds.

3.4 Emulator / device setup (all platforms)

List available devices:

`flutter devices`

Create an Android emulator (if none shows up): 1. Open Android Studio → **More Actions** → **Virtual Device Manager**. 2. Click **Create Device**, pick a phone profile (e.g. Pixel 6), pick a system image (a recent API level with Google Play), and finish. 3. Launch it from the Virtual Device Manager, or run `flutter emulators --launch <emulator_id>`.

Using a physical device instead: - Android: enable **Developer Options** (tap Build Number 7 times in Settings → About Phone), then enable **USB Debugging**. Connect via USB and accept the debugging prompt on the device. - iOS (Mac only): connect via USB, trust the computer on the device, and select your device as the run target in Xcode or `flutter run`.

3.5 Pointing the app at your backend

Once `flutter run` works, update the API base URL in the Flutter project (typically in `lib/config/` or `lib/constants/`) to match wherever your backend is running — e.g. `http://localhost:3000` for a local backend, or your deployed URL in production.

Note for physical devices: `localhost` refers to the phone itself, not your computer. Use your computer's local network IP address (e.g. `http://192.168.1.42:3000`) instead, and make sure both devices are on the same Wi-Fi network.

4. .env Reference

Each service in this monorepo has its own .env file, copied from a matching .env.example. Never commit real .env files — only .env.example templates.

4.1 backend/.env

Variable	What it does	Where to get it
PORT	Port the Express server listens on	Pick any free port, e.g. 3000
MONGO_URI	Connection string for MongoDB	Section 2.4 (Atlas) or the local Docker URI (mongodb://localhost:27018)
JWT_SECRET	Signing secret for auth tokens	Generate a long random string yourself — never reuse a tutorial's example value
NODE_ENV	Runtime mode	development locally, production when deployed
CORS_ORIGIN	Which frontend origin is allowed to call the API	Your frontend's URL, e.g. http://localhost:5173

4.2 frontend/.env

Variable	What it does	Where to get it
VITE_API_URL	Base URL the frontend uses to call the backend	Wherever your backend is running, e.g. http://localhost:3000

Vite only exposes variables prefixed with VITE_ to the browser — anything else will be undefined in the frontend code.

4.3 Mobile (Flutter) config

Flutter doesn't use a .env file by convention in this template — the backend URL is set directly in a config/constants file (see Section 3.5). If your team adds secrets to the Flutter app (API keys, etc.), consider the flutter_dotenv package rather than hardcoding them.

4.4 General .env tips

- Always copy from the example file rather than creating one from scratch, so you don't miss a variable:

```
cp .env.example .env
```

- Restart the dev server after changing any `.env` value — none of these frameworks hot-reload environment variables.
- If a variable “isn’t working,” check for trailing spaces, missing quotes around values with special characters, and that the file is actually named `.env` (not `.env.txt`).